

Lab 02: Architectural Decoupling & Cloud Deployment

Transform a monolithic Streamlit application (`logic.py`) into a cleanly separated, modular system utilizing a Service-Oriented architecture pattern, manage environment secrets safely, and publish the final product to a cloud production environment.

The Target Multi-File Architecture

Students must break down the single monolithic file into the following precise structure:

```
1  |  📁 review_analytics_platform/
2  |  |  📁 database/
3  |  |  |  db_manager.py      # Manages SQLite configurations, schemas, and CRUD executions
4  |  |  📁 services/
5  |  |  |  gemini_service.py  # Manages Google GenAI connection, formatting payloads, and
6  |  |  |  parsing metrics
7  |  |  .env                  # Private local developer runtime variable parameters (GIT-
8  |  |  IGNORED!)
9  |  |  .gitignore             # Safeguard exclusion configuration tracker rules
10 |  |  config.py              # Application wide environment loading coordinator gateway
11 |  |  requirements.txt        # Host image software environment configuration blueprint
12 |  |  app.py                 # Streamlit visual interface presentation layout controller
```

Part 1: The Code Separation Tasks

Students must distribute the functionalities of the monolithic code according to the instructions below.

1. config.py & .env (Central Configuration Layer)

- **Task:** Isolate configuration values away from functional code execution blocks.
- **Requirements:**
 - Create a `.env` file containing variables for `GEMINI_API_KEY`, `MODEL_NAME`, and `DB_NAME`.
 - Write `config.py` to use `load_dotenv()` and safely load these parameters into global runtime flags using `os.getenv()`.

- Include a conditional fallback or safety gate check that explicitly halts processing (`sys.exit(1)`) if `GEMINI_API_KEY` evaluates to `None`.

2. database/db_manager.py (Persistence Layer)

- **Task:** Extract every function executing an automated SQL string operation.
- **Requirements:** Move and expose four dedicated, pure functions using parameter bindings to prevent SQL injection vulnerabilities:
 - `init_db()`: Sets up the structural SQLite schema table layout.
 - `save_summary(filename, summary, rating, category)`: Writes records to disk.
 - `get_summaries_by_category(category)`: Fetches historical lists filtered by tier category.
 - `get_summary_by_id(summary_id)`: Fetches a single record context entry matching a primary row key ID.

3. services/gemini_service.py (Infrastructure/API Layer)

- **Task:** Isolate external cloud API network interactions and prompt instructions.
- **Requirements:**
 - Initialize the Google GenAI client object within this module using the key passed down from `config.py`. You may also use other free AI model.
 - Expose an operational wrapper function `analyze_review_sentiment(review_content)` that takes text strings, queries the Gemini Cloud model with specific system instructions, extracts the rating, determines the rating classification container category (Good/Average/Bad), and returns these parsed elements to the caller.

4. app.py (Presentation & UI Controller Layer)

- **Task:** Act strictly as the user interface layout skin.
- **Requirements:**
 - Remove all raw SQL blocks and direct `Client` API declarations.
 - Import modules from your custom database and services subfolders.
 - Control layout grids using `st.columns`, capture runtime file drag-and-drops via `st.file_uploader`, manage interface tabs using `st.session_state`, and orchestrate workflow execution by delegating the computational heavy lifting to your background service functions.

Part 2: Submission Instructions

Once you have completed your code refactoring, complete the following two deployment steps:

1. **GitHub:** Upload all your refactored project files and folders to a GitHub repository (ensure your `.env` and `.db` files are ignored!).

2. **Streamlit Cloud:** Host your application live on Streamlit Cloud, making sure to add your GEMINI_API_KEY to the Advanced Secrets panel.

What to Submit:

Create and submit a single plain text file named `lab02_yourname.txt` using the exact layout below. Do **not** upload your raw python files directly to Moodle.

Plaintext

```
1  =====
2  STUDENT NAME      : [Enter Your Full Name Here]
3  LAB ASSIGNMENT    : Lab 02
4  =====
5
6  [GITHUB REPOSITORY LINK]
7  https://github.com/your-username/your-repository-name
8
9  [STREAMLIT LIVE APPLICATION LINK]
10 https://your-custom-subdomain.streamlit.app/
11
12 =====
```